# *Python Comprehensions*

A complete reference — syntax, intent, nested patterns, and when to use each.

✎ Click any text to edit this document

## *The two-part anatomy*

Every comprehension is built from the same two conceptual parts, regardless of type. Orange is **what you want** — the expression evaluated for each item. Blue is **the way to get there** — the loop that drives iteration.

```
[ i**2 for i in range(5) ]
```

What you want | The way to get there

```
→ [0, 1, 4, 9, 16]
```

## *The six comprehension types*

1 — List comprehension

```
[ x**2 for x in range(6) ]
```

What you want | The way to get there

```
→ [0, 1, 4, 9, 16, 25]
```

> Produces an ordered, mutable list. The most common comprehension. Use whenever you need a sequence you can index, slice, or append to later.

2 — List comprehension with filter

```
[ x**2 for x in range(6) if x % 2 == 0 ]
```

What you want | The way to get there | Filter — items that fail are dropped

→ `[0, 4, 16]`

> The trailing **if** acts as a gate. Items that don't pass the condition are dropped entirely — the output list is shorter than the input. No **else** is allowed here: there is nothing to put in place of a dropped item.

3 — List comprehension with ternary (if before for)

```
[ x**2 if x%2==0 else 0 for x in range(6) ]
```

What you want (with else branch) | The way to get there

→ `[0, 0, 4, 0, 16, 0]`

> The **if** before the **for** is a ternary expression — `value_if_true if condition else value_if_false`. Every item produces exactly one output: the output list is always the same length as the input. The **else** is mandatory.

4 — Dict comprehension

```
{ x: x**2 for x in range(5) }
```

key: value pair | The way to get there

→ `{0: 0, 1: 1, 2: 4, 3: 9, 4: 16}`

> Uses curly braces with a **key: value** expression. Produces a dictionary — order is preserved in Python 3.7+. Useful for building lookup tables, inverting a mapping, or grouping data by a computed key.

5 — Set comprehension

```
{ x % 3 for x in range(9) }
```

What you want | The way to get there

→ `{0, 1, 2}` — duplicates removed automatically

> Curly braces without a colon produce a set. Duplicates are collapsed — you get only unique values. Order is not guaranteed. Use when you care about membership, not position or count.

6 — Generator expression

```
( x**2 for x in range(6) )
```

What you want | The way to get there

→ a lazy iterator — yields one value at a time, nothing stored in memory

> Parentheses instead of brackets. Values are computed on demand — the entire sequence is never held in memory. Ideal inside `sum()`, `max()`, `any()`, `all()`, or anywhere you consume a sequence once and discard it. For large datasets, this can be the difference between 4 MB and 4 GB of RAM.

---

## *The `if` placement rule*

Where you place the `if` completely changes what it does. This is one of the most commonly confused points in Python comprehensions.

AFTER THE FOR — FILTER

```
[ x**2 for x in range(6) if x%2==0 ]
```

→ `[0, 4, 16]` — list shrinks

> Items that fail the condition **vanish entirely**. No **else** allowed — there's nothing to substitute. Use when you want a subset.

→ Use for: selecting a subset of items

BEFORE THE FOR — TERNARY

```
[ x**2 if x%2==0 else 0 for x in range(6) ]
```

→ `[0, 0, 4, 0, 16, 0]` — same length

> Every item produces exactly one output. **else is mandatory**. Use when you want to transform items differently but keep all of them.

→ Use for: branching transforms, keeping all items

**Memory trick:** if the `if` has an `else`, it belongs at the **front** (it's part of the expression). If it has no `else`, it belongs at the **back** (it's a filter). You can also combine both in the same comprehension: `[x**2 if x>0 else 0 for x in range(-3,4) if x ≠ 2]` — the ternary shapes the value, the trailing filter drops items entirely.

---

## *Nested `for` loops — five patterns*

Multiple `for` clauses execute left to right, exactly like nested loops written longhand. The key question is always: *where does the inner comprehension live — as a second clause, or as the expression itself?*

Case 1 — Flatten: 2D → 1D

```
matrix = [[1,2,3],[4,5,6],[7,8,9]]

[ n  for row in matrix  for n in row ]
```

What you want | Outer loop | Inner loop

→ `[1, 2, 3, 4, 5, 6, 7, 8, 9]`

> Two **for** clauses at the top level. The outer loop provides rows; the inner loop unpacks each row into individual elements. Structure is destroyed — you get a single flat list. Row boundaries are lost.

→ Use when: you want to dissolve the 2D structure and work with all elements as a single sequence.

Case 2 — Transform every element, preserve shape

```
matrix = [[1,2,3],[4,5,6],[7,8,9]]

[ [n*2 for n in row]  for row in matrix ]
```

Inner comprehension = expression | Outer loop (one per row)

→ `[[2,4,6], [8,10,12], [14,16,18]]`

> The inner comprehension **is the expression**, not a second clause. The outer loop iterates rows; the inner comprehension produces a new list per row. Same number of rows, same number of columns — shape is fully preserved.

→ Use when: you want to apply an operation to every element without changing the structure.

Case 3 — Filter elements within rows, preserve shape

```
matrix = [[1,2,3],[4,5,6],[7,8,9]]

[ [n for n in row  if n%2==0]  for row in matrix ]
```

Inner comprehension with filter | Outer loop | Filter inside inner

→ `[[2], [4, 6], [8]]`

> Same structure as Case 2, but the inner comprehension filters. Rows stay as rows but may shrink unevenly. Useful when rows are meaningful groups (e.g. per-user records, per-day events) and you want to remove items **within** each group independently.

→ Use when: you want to filter within each sublist while keeping rows as separate entities.

Case 4 — Filter elements and flatten

```
matrix = [[1,2,3],[4,5,6],[7,8,9]]

[ n | for row in matrix | for n in row | if n%2==0 ]
```

What you want · Outer loop · Inner loop · Filter on individual elements

→ `[2, 4, 6, 8]`

> Two **for** clauses flatten first, then a trailing **if** filters individual elements. Row identity is lost. Use when you want a flat list of only specific elements from across all sublists and you don't care which row they came from.

→ Use when: you want a flat collection of selected elements, row boundaries don't matter.

Case 5 — Filter whole rows, preserve shape

```
matrix = [[1,2,3],[4,5,6],[7,8,9]]

[ row | for row in matrix | if sum(row) > 10 ]
```

Whole row as expression · One loop over rows · Row-level filter

→ `[[4, 5, 6], [7, 8, 9]]`

> Only one **for** clause, iterating rows. The filter tests the row as a whole unit. Surviving rows are kept intact — their elements are untouched. Result is still 2D, just with fewer rows.

→ Use when: you want to select or discard entire sublists based on a row-level property.

## *Quick decision table*

| INTENT | PATTERN | OUTPUT SHAPE |
|---|---|---|
| `Ordered sequence` | `[expr for x in it]` | List, same length as input |
| `Unique values only` | `{expr for x in it}` | Set, deduplicated |

| INTENT | PATTERN | OUTPUT SHAPE |
|---|---|---|
| Key-value lookup | `{k: v for x in it}` | Dict |
| Memory-efficient pipeline | `(expr for x in it)` | Generator, lazy |
| Keep subset only | `[expr for x in it if cond]` | List, shorter than input |
| Branch transform, keep all | `[a if c else b for x in it]` | List, same length as input |
| Flatten 2D → 1D | `[n for r in m for n in r]` | Flat list |
| Transform, keep shape | `[[f(n) for n in r] for r in m]` | 2D list, same shape |
| Filter within rows | `[[n for n in r if c] for r in m]` | 2D list, rows may shrink |
| Filter whole rows | `[row for row in m if row_cond]` | 2D list, fewer rows |

## *Notes*

**Cognitive load.** Oneliners are generally easier to read, requiring less mental juggling to reassemble bits and pieces — but only once you've internalised the pattern. Beginners often find the for loop clearer because it matches a procedural mental model. The comprehension reads as a declaration of intent; the loop reads as a sequence of instructions.

**Readability ceiling.** The moment a comprehension needs more than one filter and a nested for, consider breaking it into a regular loop. Python's own style guide (PEP 8) suggests that comprehensions should be simple enough to read in one pass.

**Generators vs lists.** Prefer a generator expression when you only need to consume the result once — inside `sum()`, `join()`, `any()`, etc. Convert to a list only when you need to index it, iterate it multiple times, or check its length.

**The two-clause rule for nested fors.** If the inner `for` appears as a second top-level clause, the structure flattens. If it appears as an inner comprehension inside the expression, the structure is preserved. This is the single most important distinction in nested comprehensions.